Some Notes on Assignment 2

VPC configuration (step 3)

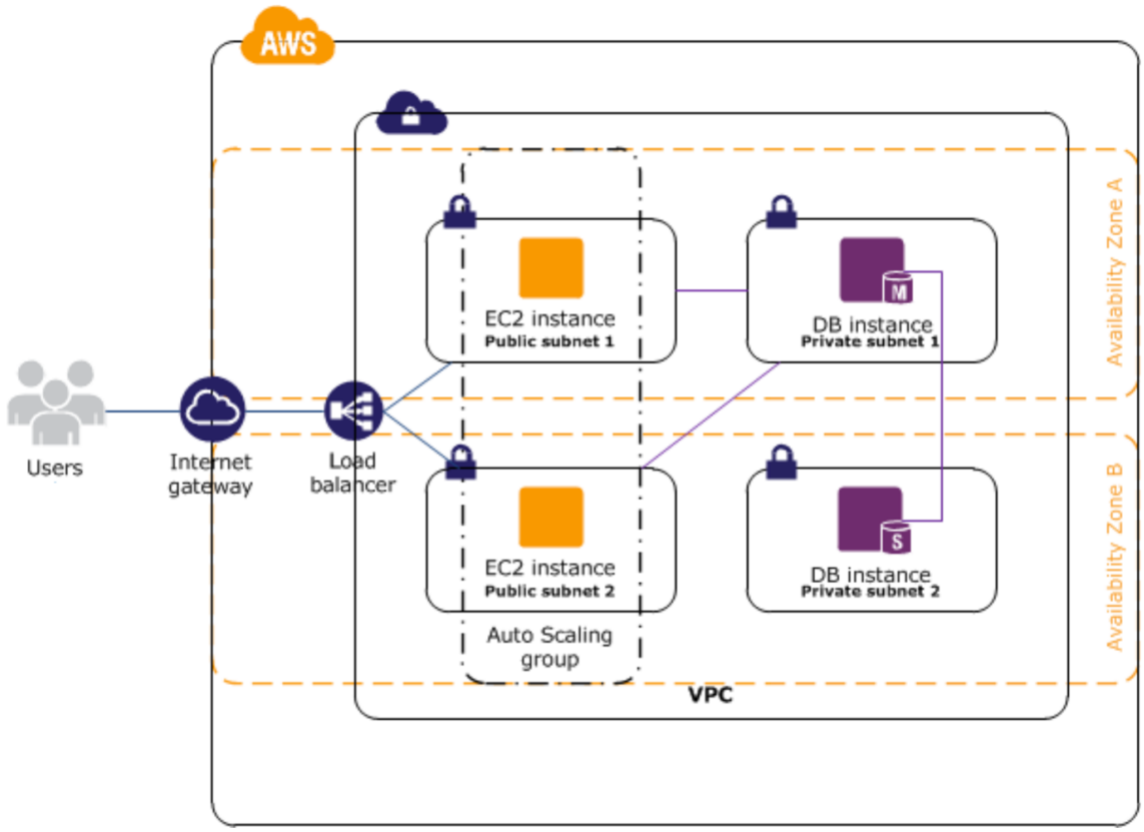
Step 3 of Assignment 2 specifies:

"Creation of a VPC with subnets into which your application will be deployed. Creation of suitable security groups."

The following is a clarification of exactly what is required, especially if you choose not to deploy a database on a private instance as part of your assignment.

- You are required to create a VPC.
- This VPC should have public and optionally private subnets in at least two (ideally three) different availability zones.
- Your web server instances should run in this VPC.
- If you deploy a database instance, it should be in a private subnet. We don't explicitly require you to deploy a database instance; this is "additional functionality". You can use a public database service or have no database at all.
- Security groups are per-VPC. For those without a database instance, "creation of suitable security groups" just means security groups for your load balancer and public instances.
- When you create a public subnet, there is an *Auto-assign public IPv4 address* setting which is set to **No** unless you change it. This is in contrast with the default public subnets in your default VPC which have this turned on. You will need to change this ("Modify auto-assign IP settings") to be able to route traffic directly to your servers for testing.

The picture below shows a typical web application architecture.



Auto-scaling policy (step 5)

Please note that you are required to use either *Simple Scaling* or *Step Scaling* and **not** Target Tracking for your dynamic scaling policy.

Generation of test traffic and load balancer demo (step 6)

To help you generate test traffic to your load balancer, you could use a tool such as **curl** to send a sequence of requests to it, each with a slightly different URL and have a look at the server logs afterwards. For example this command would send 100 requests to *http://lbdnsname/1*, *http://lbdnsname/2*, etc.

curl -s "http://lbdnsname/?[1-100]"

Then for step 6, have a look at the logs on each of the target instances. You can find the apache access log at **/var/log/httpd/access_log**.

The **tail** command with the **-f** option is handy for reading "live" logs. (i.e. use **tail -f**) and you should see how these requests are distributed.

Role for pushing custom metrics from EC2 to CloudWatch (step 7)

The Week 9 lab on pushing metrics to CloudWatch service involves running a script on an EC2 instance that uses the AWS CLI. Normally the CLI needs API credentials to access the user's AWS account. However this is problematic in our Learner Lab setup as the credentials change frequently change, so we cannot set them up on an AMI or in User Data on a Launch Template.

The solution is to use an *IAM role* that is provided in the Learner Lab. The role is called **LabInstanceProfile** and can be granted to an instance at launch time:

- For auto-scaled instances, this option is available in the Launch Template, expand *Advanced details*, and under *IAM instance profile* select *LabInstanceProfile*.
- For a manually launched instance, this appears as *IAM role* in Step 3 of the legacy launch wizard. For the newer launch wizard, it is under *Advanced details.*

One additional thing is required – you need to specify the region. To set a default region, you can run *aws configure* on your master instance and set the region to *us-east-1*, leaving the other fields blank; i.e.

```
$ aws configure
AWS Access Key ID [None]:
AWS Secret Access Key [None]:
Default region name [None]: us-east-1
Default output format [None]:
```

Alternatively you can add `--region "us-east-1"` to each CLI command in your script.

Then you can create a new AMI from the updated master instance that will have the region specification built in.

If you'd prefer not to edit the master instance and create a new AMI, you can alternatively add User data to your launch template to set the region.